

2-Layer MLP with Sigmoid Activation

Mahmudul Hasan

February 18, 2026

1 Definitions

1.1 Multilayer Perceptron

A Multilayer Perceptron (MLP) is a type of feedforward artificial neural network that consists of multiple layers of nodes. Each layer is fully connected to the next one, and the nodes in each layer apply a nonlinear activation function to their inputs. MLPs are commonly used for classification and regression tasks.

1.2 Sigmoid Activation Function

The sigmoid activation function is defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

The derivative of the sigmoid is given by:

$$\sigma'(x) = \frac{e^{-x}}{(1 + e^{-x})^2} = \sigma(x) (1 - \sigma(x))$$

2 2-Layer MLP

2.1 Network Architecture

A 2-layer MLP with sigmoid activation consists of:

- Input layer: n nodes ($x_1, x_2, \dots, x_n$)
- Hidden layer: m nodes (pre-activation $z_1, z_2, \dots, z_m$, post-activation $a_1, a_2, \dots, a_m$) with sigmoid activation
- Output layer: 1 node (pre-activation z_o , output o) with sigmoid activation

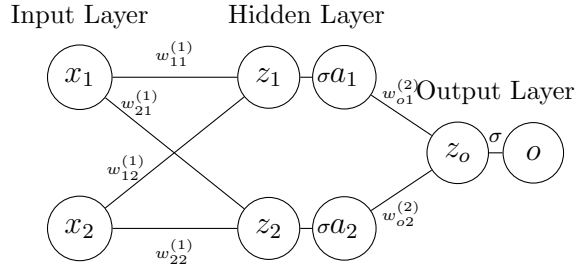


Figure 1: Architecture of a 2-layer MLP (shown for $n = 2$, $m = 2$ example)

Here, $w_{ji}^{(1)}$ represents the weight from the i -th input node to the j -th hidden node (pre-activation), and $w_{oj}^{(2)}$ represents the weight from the j -th hidden node (post-activation) to the output pre-activation node. The sigmoid activation function σ is applied after each layer.

2.2 Forward Pass

In general, the forward pass for each hidden node j (where $j = 1, 2, \dots, m$) is:

$$z_j = \sum_{i=1}^n w_{ji}^{(1)} x_i + b_j^{(1)}$$

$$a_j = \sigma(z_j)$$

And for the single output node:

$$z_o = \sum_{j=1}^m w_{oj}^{(2)} a_j + b_o^{(2)}$$

$$o = \sigma(z_o)$$

In general matrix form, let the input vector be $\mathbf{x} = [x_1 x_2 \dots x_n]^T \in \mathbb{R}^{n \times 1}$ (column vector). The weights for the hidden layer are represented as a matrix $\mathbf{W}^{(1)} \in \mathbb{R}^{m \times n}$, where row j contains all weights going to hidden node j . With our indexing convention where $w_{ji}^{(1)}$ connects input i to hidden j , the weight matrix has elements $[\mathbf{W}^{(1)}]_{ji} = w_{ji}^{(1)}$. The pre-activation output of the hidden layer can be calculated as:

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)}$$

where $\mathbf{b}^{(1)} \in \mathbb{R}^m$ is the bias vector for the hidden layer. The post-activation output is:

$$\mathbf{a}^{(1)} = \sigma(\mathbf{z}^{(1)})$$

The pre-activation output of the network can then be calculated as:

$$z_o = \mathbf{w}^{(2)} \mathbf{a}^{(1)} + b_o^{(2)}$$

where $\mathbf{w}^{(2)} = [w_{o1}^{(2)}, w_{o2}^{(2)}, \dots, w_{om}^{(2)}]^T \in \mathbb{R}^m$ is the weight vector for the output layer and $b_o^{(2)}$ is the bias scalar for the single output node. Finally, the output is:

$$o = \sigma(z_o)$$

2.3 Loss Function

The loss is calculated over a batch of N training examples using Mean Squared Error (MSE):

$$\mathcal{L} = \frac{1}{N} \sum_{k=1}^N (y^{(k)} - o^{(k)})^2$$

where $y^{(k)}$ is the true label and $o^{(k)}$ is the predicted output for the k -th training example in the batch. The superscript (k) denotes the sample index to distinguish it from node indices.

2.4 Gradients of the Loss Function

To update the weights during training, we need to compute the gradients of the loss function with respect to the weights. Using backpropagation, we can derive the gradients as follows:

Gradient for output layer weights:

For each weight connecting hidden node j (where $j = 1, 2, \dots, m$) to the output, we apply the chain rule:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_{oj}^{(2)}} &= \frac{\partial \mathcal{L}}{\partial o} \cdot \frac{\partial o}{\partial z_o} \cdot \frac{\partial z_o}{\partial w_{oj}^{(2)}} \\ &= \frac{2}{N} (o - y) \cdot \sigma'(z_o) \cdot a_j \end{aligned}$$

Gradient for output layer bias:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial b_o^{(2)}} &= \frac{\partial \mathcal{L}}{\partial o} \cdot \frac{\partial o}{\partial z_o} \cdot \frac{\partial z_o}{\partial b_o^{(2)}} \\ &= \frac{2}{N} (o - y) \cdot \sigma'(z_o) \end{aligned}$$

Gradient for hidden layer weights:

For the hidden layer, we need to backpropagate through the output layer. Since there is only one output node, the gradient for each weight $w_{ji}^{(1)}$ (where $i = 1, 2, \dots, n$ and $j = 1, 2, \dots, m$) is:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_{ji}^{(1)}} &= \frac{\partial \mathcal{L}}{\partial o} \cdot \frac{\partial o}{\partial z_o} \cdot \frac{\partial z_o}{\partial a_j} \cdot \frac{\partial a_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ji}^{(1)}} \\ &= \frac{2}{N} (o - y) \cdot \sigma'(z_o) \cdot w_{oj}^{(2)} \cdot \sigma'(z_j) \cdot x_i \end{aligned}$$

where $\frac{\partial z_o}{\partial a_j} = w_{oj}^{(2)}$ (from $z_o = \sum_{j=1}^m w_{oj}^{(2)} a_j + b_o^{(2)}$) and $\frac{\partial z_j}{\partial w_{ji}^{(1)}} = x_i$ (from $z_j = \sum_{i=1}^n w_{ji}^{(1)} x_i + b_j^{(1)}$).

Gradient for hidden layer biases:

For each hidden layer bias $b_j^{(1)}$ (where $j = 1, 2, \dots, m$):

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial b_j^{(1)}} &= \frac{\partial \mathcal{L}}{\partial o} \cdot \frac{\partial o}{\partial z_o} \cdot \frac{\partial z_o}{\partial a_j} \cdot \frac{\partial a_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial b_j^{(1)}} \\ &= \frac{2}{N} (o - y) \cdot \sigma'(z_o) \cdot w_{oj}^{(2)} \cdot \sigma'(z_j) \end{aligned}$$